



Universidad Tecnológica
del Norte de Guanajuato

Organismo Público Descentralizado del Gobierno del Estado

ÁREA ACADÉMICA: TECNOLOGÍAS
DE LA INFORMACIÓN Y
COMUNICACIÓN

PROGRAMA EDUCATIVO:
INGENIERÍA EN DESARROLLO Y
GESTIÓN DE SOFTWARE

ASIGNATURA
APLICACIONES WEB
PROGRESIVAS

DOCUMENTACIÓN

INTEGRACION DE CLIENTES

González Avalos César Fernando

No.Control: 1223100745

**PROYECTO
INTEGRADOR**
ONBOARDING DE
CLIENTES
PERSONAS
FÍSICAS

GRUPO: GIDS6103

El presente documento describe la solución técnica implementada para el Proyecto Integrador: Onboarding de Clientes Personas Físicas. El objetivo principal es desarrollar una aplicación en Java que permita registrar clientes personas físicas, validar su información personal y financiera, crear una cuenta bancaria asociada y asignar un saldo inicial de forma automática.

El proyecto implementa buenas prácticas de desarrollo de software, persistencia de datos con PostgreSQL, validaciones de negocio estrictas, seguridad basada en tokens JWT y un diseño arquitectónico por capas que separa claramente las responsabilidades del sistema.

HISTORIA DE USUARIO

"Como ejecutivo de una institución financiera, necesito registrar clientes personas físicas en el sistema para asignarles una cuenta bancaria y permitirles realizar operaciones financieras."

Esta historia de usuario se traduce en un flujo de onboarding completo que abarca desde el registro inicial del cliente hasta la creación automática de su cuenta bancaria y sus credenciales de acceso al sistema.

ARQUITECTURA DE LA SOLUCIÓN

La aplicación sigue una arquitectura de capas (Layered Architecture) con el patrón MVC, que es el estándar de la industria para aplicaciones empresariales en Spring Boot:

Capa	Responsabilidad	Paquete Java
Controller	Expone endpoints REST, recibe peticiones HTTP y delega al servicio	controller.onboarding
Service	Contiene toda la lógica de negocio, validaciones y orquestación	service.onboarding.impl
Repository	Acceso a datos con JPA/Hibernate. Queries personalizados	repository.onboarding
Entity	Mapeo ORM de las tablas de la base de datos	entity.onboarding
DTO	Objetos de transferencia para request/response (desacoplamiento)	model.request / model.response
Mapper	Conversión entre entidades y DTOs usando MapStruct	mapper.onboarding
Exception	Excepciones personalizadas y manejo global centralizado	exception.onboarding
Config	Seguridad (JWT, Spring Security), Swagger, Flyway, HikariCP	config

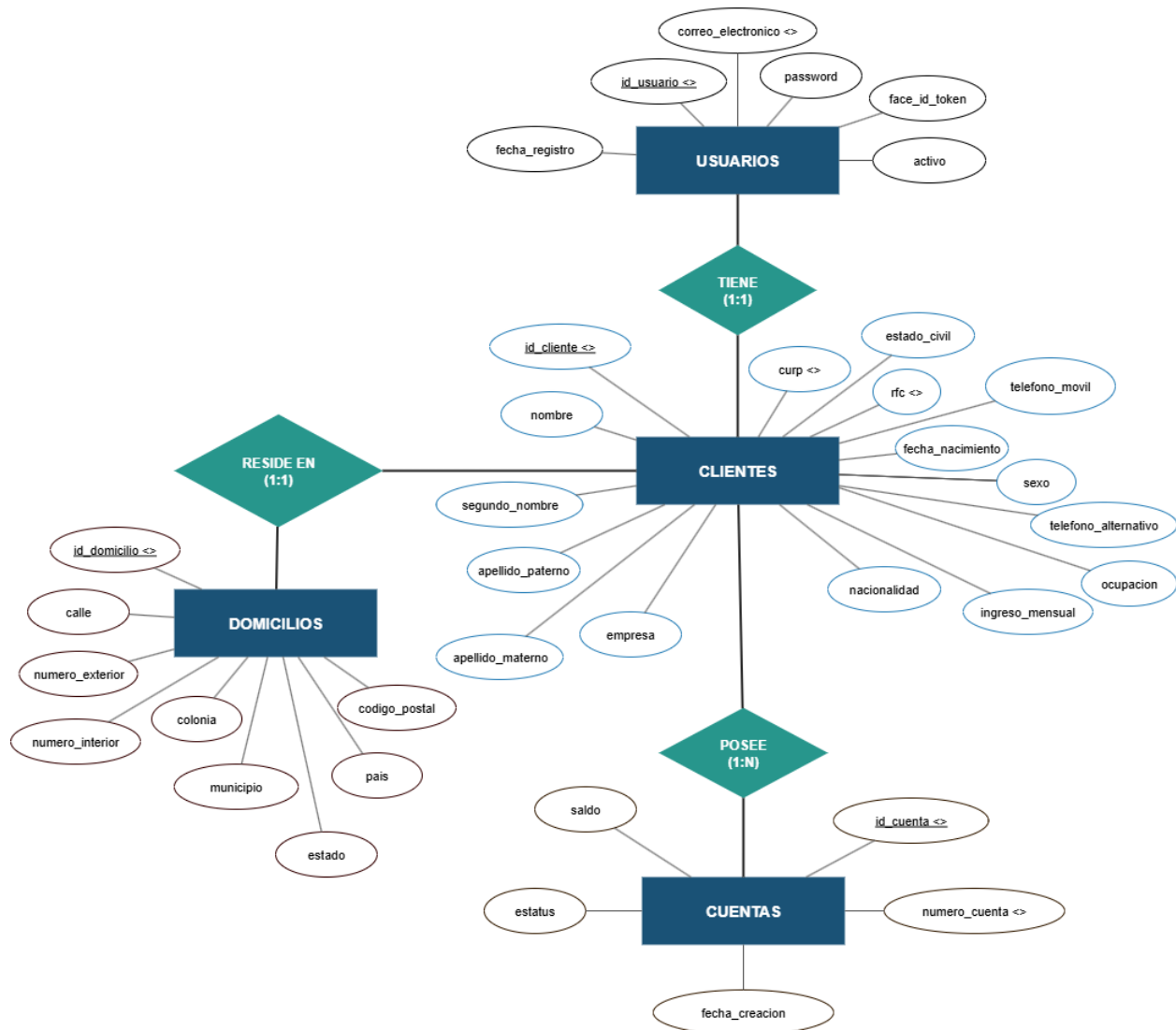
La comunicación entre capas es estrictamente unidireccional: Controller - Service - Repository. Esto garantiza la separación de responsabilidades y facilita el testing unitario e integración.

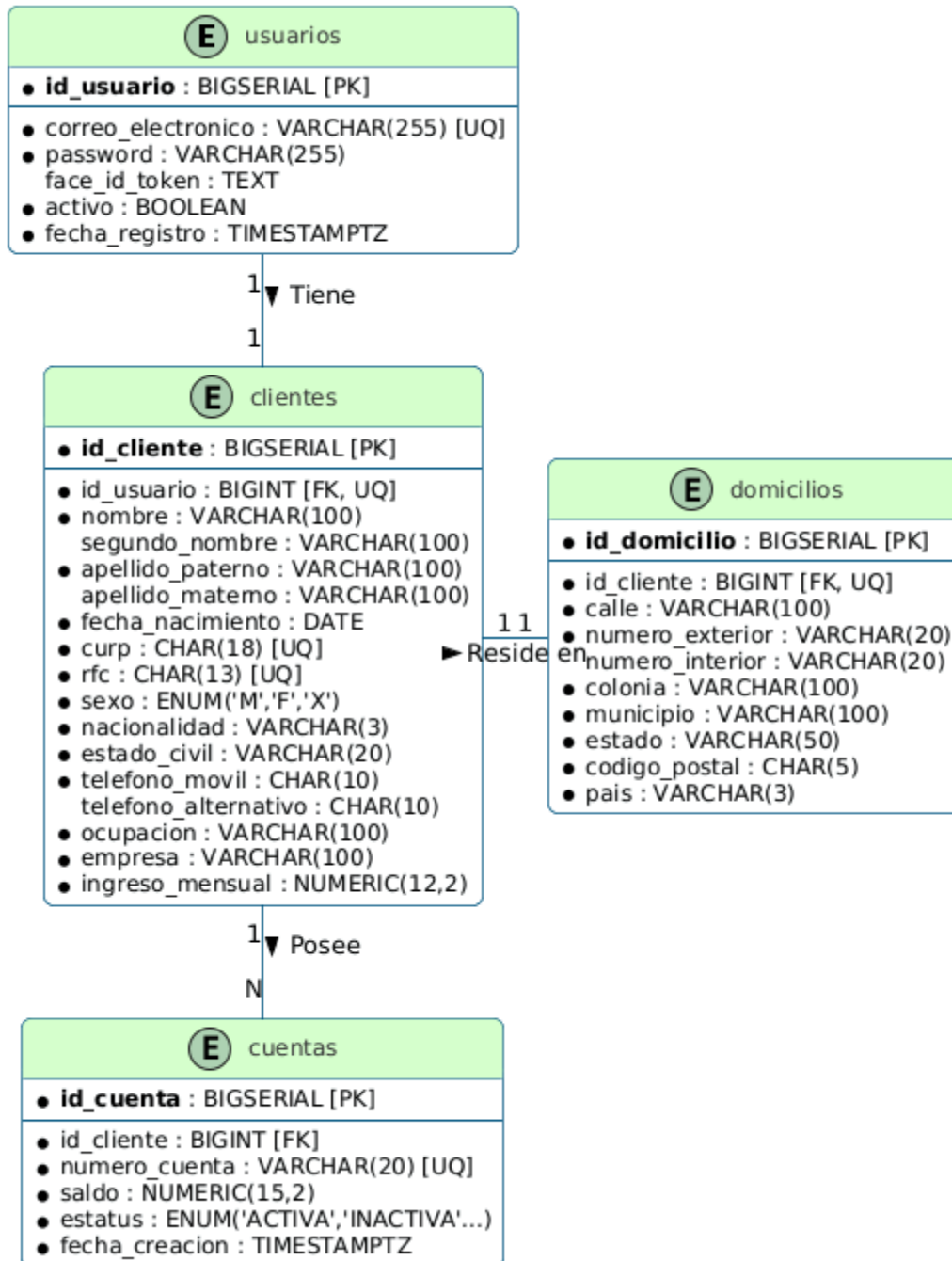
DISEÑO DE BASE DE DATOS

Diagrama Entidad-Relación

El modelo de datos está compuesto por 4 tablas principales con las siguientes relaciones:

1. Usuario (1) - (1) Cliente: Relación uno a uno. Cada cliente tiene exactamente un usuario de acceso.
2. Cliente (1) - (1) Domicilio: Relación uno a uno. Cada cliente tiene exactamente un domicilio registrado.
3. Cliente (1) - (N) Cuenta: Relación uno a muchos. Un cliente puede tener múltiples cuentas bancarias.





Script de Creación (Migración)

La base de datos se gestiona mediante migraciones versionadas de Flyway, lo que garantiza reproducibilidad y control de versiones del esquema. El archivo de migración es:

V5__create_onboarding_schema.sql

SCRIPT

```
CREATE TYPE estatus_cuenta AS ENUM ('ACTIVA', 'INACTIVA', 'BLOQUEADA', 'CANCELADA');
CREATE TYPE sexo_biologico AS ENUM ('M', 'F');

CREATE TABLE usuarios (
    id_usuario BIGSERIAL PRIMARY KEY,
    correo_electronico VARCHAR(255) NOT NULL UNIQUE
        CHECK (correo_electronico ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'),
    password VARCHAR(255) NOT NULL,
    face_id_token TEXT,
    activo BOOLEAN DEFAULT TRUE NOT NULL,
    fecha_registro TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL
);

CREATE TABLE clientes (
    id_cliente BIGSERIAL PRIMARY KEY,
    id_usuario BIGINT NOT NULL UNIQUE,
    nombre VARCHAR(100) NOT NULL,
    segundo_nombre VARCHAR(100),
    apellido_paterno VARCHAR(100) NOT NULL,
    apellido_materno VARCHAR(100),
    fecha_nacimiento DATE NOT NULL,
    curp CHAR(18) NOT NULL UNIQUE CHECK (curp ~ '^[A-Z0-9]{18}$'),
    rfc CHAR(13) NOT NULL UNIQUE CHECK (rfc ~ '^[A-Z0-9]{13}$'),
    sexo sexo_biologico NOT NULL,
    nacionalidad VARCHAR(3) NOT NULL DEFAULT 'MEX',
    estado_civil VARCHAR(20) NOT NULL
        CHECK (estado_civil IN ('SOLTERO', 'CASADO', 'DIVORCIADO', 'VIUDO', 'CONCUBINATO')),
    telefono_movil CHAR(10) NOT NULL CHECK (telefono_movil ~ '^[0-9]{10}$'),
    telefono_alternativo CHAR(10) CHECK (telefono_alternativo ~ '^[0-9]{10}$'),
    ocupacion VARCHAR(100) NOT NULL,
    empresa VARCHAR(100) NOT NULL,
    ingreso_mensual NUMERIC(12, 2) NOT NULL CHECK (ingreso_mensual > 0),
    CONSTRAINT fk_cliente_usuario
        FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario) ON DELETE CASCADE
);

CREATE TABLE domicilios (
    id_domicilio BIGSERIAL PRIMARY KEY,
    id_cliente BIGINT NOT NULL UNIQUE,
    calle VARCHAR(100) NOT NULL,
    numero_exterior VARCHAR(20) NOT NULL,
    numero_interior VARCHAR(20),
    colonia VARCHAR(100) NOT NULL,
    municipio VARCHAR(100) NOT NULL,
    estado VARCHAR(50) NOT NULL,
    codigo_postal CHAR(5) NOT NULL CHECK (codigo_postal ~ '^[0-9]{5}$'),
    pais VARCHAR(3) NOT NULL DEFAULT 'MEX',
    CONSTRAINT fk_domicilio_cliente
        FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente) ON DELETE CASCADE
);

CREATE TABLE cuentas (
    id_cuenta BIGSERIAL PRIMARY KEY,
    id_cliente BIGINT NOT NULL,
    numero_cuenta VARCHAR(20) NOT NULL UNIQUE CHECK (numero_cuenta ~ '^[0-9]+$'),
    saldo NUMERIC(15, 2) DEFAULT 0.00 NOT NULL CHECK (saldo >= 0),
    estatus estatus_cuenta DEFAULT 'ACTIVA' NOT NULL,
```

```

    fecha_creacion TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_cuenta_cliente
        FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente) ON DELETE RESTRICT
);
    
```

DETALLE DE TABLAS Y TIPOS DE DATOS

A continuación se detalla cada tabla con la justificación técnica de los tipos de datos seleccionados, siguiendo mejores prácticas de rendimiento y uso de memoria:

Tabla: usuarios

Campo	Tipo de Dato	Restricciones	Justificación
id_usuario	BIGSERIAL	PRIMARY KEY	Autoincremental 64-bit para escalabilidad
correo_electronico	VARCHAR(255)	NOT NULL, UNIQUE, CHECK (regex)	RFC 5321 permite hasta 254 caracteres
password	VARCHAR(255)	NOT NULL	BCrypt genera hashes de ~60 caracteres
face_id_token	TEXT	NULL	Tokens biométricos pueden ser extensos
activo	BOOLEAN	NOT NULL, DEFAULT TRUE	1 byte en PostgreSQL, altamente eficiente
fecha_registro	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Obligatorio en sistemas financieros (zona horaria)

Tabla: clientes

Campo	Tipo de Dato	Restricciones	Justificación
id_cliente	BIGSERIAL	PRIMARY KEY	Autoincremental de 64 bits
id_usuario	BIGINT	NOT NULL, UNIQUE, FK	Relación 1:1 con usuarios
nombre	VARCHAR(100)	NOT NULL	Nombres latinoamericanos pueden ser largos
segundo_nombre	VARCHAR(100)	NULL	Campo opcional
apellido_paterno	VARCHAR(100)	NOT NULL	Apellidos compuestos requieren espacio
apellido_materno	VARCHAR(100)	NULL	Puede no existir legalmente
fecha_nacimiento	DATE	NOT NULL	3 bytes, sin hora innecesaria
curp	CHAR(18)	NOT NULL, UNIQUE, CHECK	Fijo: CHAR ahorra bytes vs VARCHAR
rfc	CHAR(13)	NOT NULL, UNIQUE, CHECK	Persona física son exactamente 13 chars
sexo	sexo_biologico (ENUM)	NOT NULL	Catálogo: M, F, X. Integridad garantizada
nacionalidad	VARCHAR(3)	NOT NULL, DEFAULT MEX	Código ISO 3166-1 alpha-3
estado_civil	VARCHAR(20)	NOT NULL, CHECK	Catálogo validado a nivel DB

telefono_movil	CHAR(10)	IN(...) NOT NULL, CHECK (regex)	Fijo 10 dígitos. CHAR optimiza
telefono_alternativo	CHAR(10)	NULL, CHECK	Opcional, misma validación
ocupacion	VARCHAR(100)	NOT NULL	Texto libre controlado
empresa	VARCHAR(100)	NOT NULL	Texto libre controlado
ingreso_mensual	NUMERIC(12,2)	NOT NULL, CHECK > 0	Precisión exacta, sin errores de punto flotante

Tabla: domicilios

Campo	Tipo de Dato	Restricciones	Justificación
id_domicilio	BIGSERIAL	PRIMARY KEY	Autoincremental
id_cliente	BIGINT	NOT NULL, UNIQUE, FK	Relación 1:1 con clientes
calle	VARCHAR(100)	NOT NULL	Nombres de calle extensos
numero_exterior	VARCHAR(20)	NOT NULL	Puede contener "S/N" o letras
numero_interior	VARCHAR(20)	NULL	Campo opcional
colonia	VARCHAR(100)	NOT NULL	Colonias con nombres largos
municipio	VARCHAR(100)	NOT NULL	Municipios extensos
estado	VARCHAR(50)	NOT NULL	Entidades federativas
codigo_postal	CHAR(5)	NOT NULL, CHECK (regex)	Fijo 5 dígitos. CHAR optimiza
pais	VARCHAR(3)	NOT NULL, DEFAULT MEX	Código ISO 3166-1 alpha-3

Tabla: cuentas

Campo	Tipo de Dato	Restricciones	Justificación
id_cuenta	BIGSERIAL	PRIMARY KEY	Autoincremental
id_cliente	BIGINT	NOT NULL, FK	Relación N:1 con clientes
numero_cuenta	VARCHAR(20)	NOT NULL, UNIQUE, CHECK	Soporta CLABE (18) o internacionales
saldo	NUMERIC(15,2)	NOT NULL, CHECK >= 0, DEFAULT 0	Soporta hasta billones. Precisión exacta
estatus	estatus_cuenta (ENUM)	NOT NULL, DEFAULT ACTIVA	Catálogo: ACTIVA, INACTIVA, BLOQUEADA, CANCELADA
fecha_creacion	TIMESTAMP TZ	NOT NULL, DEFAULT NOW()	Auditoría temporal con zona horaria

IMPLEMENTACIÓN EN JAVA

Estructura del Proyecto

```

com.proyecto.servicios/
├── config/
│   ├── SecurityConfig.java      # Configuración de Spring Security
│   ├── JwtUtil.java             # Generación y validación de tokens JWT
│   ├── JwtAuthenticationFilter.java # Filtro HTTP para interceptar tokens
│   ├── RateLimitFilter.java     # Protección contra ataques DDoS (Bucket4j)
│   ├── OpenApi.java             # Configuración de Swagger con JWT
│   └── FlywayConfig.java        # Configuración de migraciones de BD
├── controller/onboarding/
│   ├── AuthController.java      # POST /auth/login
│   ├── ClienteController.java   # CRUD completo de clientes
│   ├── CuentaController.java    # Consulta de cuentas y saldo
│   └── UsuarioController.java   # Gestión de usuarios y contraseña
├── service/onboarding/
│   ├── AuthService.java         # Interface de autenticación
│   ├── ClienteService.java      # Interface de clientes
│   ├── CuentaService.java       # Interface de cuentas
│   └── impl/
│       ├── AuthServiceImpl.java # Lógica de login, JWT y contraseñas
│       ├── ClienteServiceImpl.java # Lógica de registro, validación, CRUD
│       └── CuentaServiceImpl.java # Lógica de consulta de cuentas
├── repository/onboarding/
│   ├── ClienteRepository.java   # Queries JPA personalizados
│   ├── CuentaRepository.java    # Acceso a datos de cuentas
│   └── UsuarioRepository.java    # Acceso a datos de usuarios
├── entity/onboarding/
│   ├── Usuario.java             # Entidad JPA mapeada a tabla usuarios
│   ├── Cliente.java             # Entidad JPA mapeada a tabla clientes
│   ├── Domicilio.java          # Entidad JPA mapeada a tabla domicilios
│   └── Cuenta.java              # Entidad JPA mapeada a tabla cuentas
├── model/
│   ├── request/onboarding/
│   │   ├── ClienteRegistroDTO.java # DTO de entrada para registro
│   │   ├── ClienteActualizaDTO.java # DTO de entrada para actualización
│   │   ├── LoginRequestDTO.java     # DTO de entrada para login
│   │   └── CambioPasswordDTO.java   # DTO de entrada para cambio de contraseña
│   └── response/onboarding/
│       ├── ClienteResponseDTO.java # DTO de salida con datos del cliente
│       ├── CuentaResponseDTO.java  # DTO de salida con datos de cuenta
│       ├── LoginResponseDTO.java   # DTO de salida con token JWT
│       └── UsuarioResponseDTO.java  # DTO de salida con datos del usuario
├── mapper/onboarding/
│   └── ClienteMapper.java         # Conversión Entity ↔ DTO (MapStruct)
├── exception/
│   └── GlobalExceptionHandler.java # Manejo centralizado de excepciones
└── onboarding/

```

- └─ CurpDuplicadaException.java
- └─ RfcDuplicadoException.java
- └─ CorreoDuplicadoException.java
- └─ RecursoNoEncontradoException.java
- └─ CredencialesInvalidasException.java
- └─ UsuarioInactivoException.java
- └─ ContraseñaInvalidaException.java
- └─ CuentaDesactivadaException.java

Entidades JPA

Las entidades están mapeadas con JPA/Hibernate utilizando anotaciones de Jakarta Persistence. Se utilizan relaciones bidireccionales con CascadeType.ALL para garantizar la persistencia en cascada. Las entidades emplean Lombok (@Data, @Builder, @NoArgsConstructor, @AllArgsConstructor) para reducir código boilerplate.

Entidad	Tabla	Relaciones	Características
Usuario	usuarios	1:1 con Cliente	Autenticación, BCrypt, estado activo/inactivo
Cliente	clientes	1:1 Usuario, 1:1 Domicilio, 1:N Cuenta	Datos KYC, validaciones estrictas
Domicilio	domicilios	1:1 con Cliente	Dirección completa, código postal validado
Cuenta	cuentas	N:1 con Cliente	Cuenta bancaria, saldo, estatus ENUM

Repositorios

Se implementaron repositorios extendiendo JpaRepository de Spring Data JPA, lo cual proporciona operaciones CRUD automáticas. Adicionalmente, se crearon queries personalizados con @Query (JPQL) para las consultas específicas del negocio.

Servicios (Lógica de Negocio)

La capa de servicios contiene toda la lógica de negocio. Cada servicio implementa una interfaz para respetar el principio de inversión de dependencias (DIP). Las operaciones de escritura están marcadas con @Transactional para garantizar atomicidad (si una parte del registro falla, se revierte todo automáticamente).

Controladores REST

Los controladores son la capa de exposición de la API. Reciben las peticiones HTTP, delegan al servicio correspondiente y retornan respuestas estandarizadas mediante el objeto ApiResponse, que siempre incluye: código, mensaje, path y timestamp.

Validaciones Implementadas

Las validaciones se implementan en dos niveles: a nivel de DTO (Jakarta Validation) y a nivel de base de datos (CHECK constraints). Esto garantiza un doble escudo de protección.

Campo	Validación DTO (Java)	Validación BD (PostgreSQL)
Nombre	@NotBlank, @Pattern(solo letras/espacios), @Size(2-50)	NOT NULL, VARCHAR(100)
Apellido Paterno	@NotBlank, @Pattern(solo letras/espacios), @Size(2-50)	NOT NULL, VARCHAR(100)
CURP	@NotBlank, @Size(18), @Pattern(regex oficial)	CHAR(18), UNIQUE, CHECK regex
RFC	@NotBlank, @Pattern(regex oficial mexicano)	CHAR(13), UNIQUE, CHECK regex
Correo Electrónico	@NotBlank, @Email, @Size(max=100)	VARCHAR(255), UNIQUE, CHECK regex
Contraseña	@NotBlank, @Pattern(8+ chars, mayus, minus, num, especial)	VARCHAR(255), NOT NULL
Fecha Nacimiento	@NotNull, @Past (no futura)	DATE, NOT NULL
Teléfono Móvil	@NotBlank, @Pattern(exactamente 10 dígitos)	CHAR(10), CHECK regex
Código Postal	@NotBlank, @Pattern(exactamente 5 dígitos)	CHAR(5), CHECK regex
Ingreso Mensual	@NotNull, @DecimalMin(0.01)	NUMERIC(12,2), CHECK > 0
Sexo	@NotBlank	ENUM sexo_biologico (M, F, X)
Estado Civil	@NotBlank	CHECK IN ('SOLTERO','CASADO','DIVORCIADO','VIUDO','CONCUBINATO')

Validación Avanzada: Congruencia de CURP

1. Adicionalmente, se implementó una validación de nivel avanzado que cruza la información de la CURP con los datos proporcionados por el usuario:
2. Verifica que la fecha de nacimiento codificada en la CURP (posiciones 4-9) coincida con el campo fechaNacimiento.
3. Verifica que el marcador de sexo de la CURP (posición 10: H/M) sea congruente con el campo sexo del formulario.
4. Si existe incongruencia, el registro se rechaza con un mensaje descriptivo.

Manejo de Excepciones

El sistema utiliza un `GlobalExceptionHandler` centralizado con la anotación `@ControllerAdvice` de Spring. Cada tipo de error tiene un código numérico único y un HTTP Status apropiado:

Código	Excepción	HTTP Status	Caso de Uso
2	<code>MethodArgumentNotValidException</code>	400 Bad Request	Campos obligatorios faltantes o formato inválido
3	<code>IllegalArgumentException</code>	400 Bad Request	Menor de edad, incongruencia de CURP
4	<code>CurpDuplicadaException</code>	409 Conflict	CURP ya registrada en el sistema
5	<code>RfcDuplicadoException</code>	409 Conflict	RFC ya registrado en el sistema
6	<code>CorreoDuplicadoException</code>	409 Conflict	Correo electrónico ya registrado
7	<code>DataIntegrityViolationException</code>	409 Conflict	Violación de constraint a nivel BD (conurrencia)
8	<code>RecursoNoEncontradoException</code>	404 Not Found	Cliente, usuario o cuenta no encontrada
9	<code>CredencialesInvalidasException</code>	401 Unauthorized	Contraseña incorrecta
9	<code>UsuarioInactivoException</code>	401 Unauthorized	Usuario con cuenta desactivada
9	<code>ContraseñaInvalidaException</code>	401 Unauthorized	Contraseña actual incorrecta
10	<code>CuentaDesactivadaException</code>	403 Forbidden	Usuario inactivo intenta operación de escritura
429	Rate Limit (Filtro)	429 Too Many Requests	Demasiadas peticiones por segundo (DDoS)
0	Exception (Catch-all)	500 Internal Server Error	Error inesperado del servidor

Seguridad y Autenticación (JWT)

La seguridad del sistema está implementada con Spring Security y JSON Web Tokens (JWT):

1. Las contraseñas se almacenan cifradas con `BCryptPasswordEncoder` (hashing unidireccional).
2. Al hacer login exitoso, el servidor genera un token JWT firmado con HMAC-SHA256.
3. El token tiene una vigencia de 1 hora (3600000 ms), configurable en `application.properties`.
4. Cada petición a un endpoint protegido debe incluir el header: `Authorization: Bearer <token>`.
5. El filtro `JwtAuthenticationFilter` intercepta cada request y valida el token antes de dar acceso.

Modo Solo Lectura para Cuentas Desactivadas

Cuando un cliente es dado de baja lógica, su usuario puede seguir haciendo login, pero se le asigna el rol `ROLE_READONLY`. Con este rol, el sistema le permite consultar su información

(endpoints GET), pero bloquea cualquier operación de escritura (PUT, DELETE) con un error HTTP 403 Forbidden y un mensaje claro: "Su cuenta está desactivada. Solo puede consultar información, no realizar operaciones."

ENDPOINTS DE LA API REST

Autenticación

Método	Endpoint	Descripción	Auth
POST	/auth/login	Iniciar sesión y obtener token JWT	Pública

Clientes

Método	Endpoint	Descripción	Auth
POST	/clientes	Registrar nuevo cliente (con cuenta y usuario)	Pública
GET	/clientes	Listar clientes (con filtros opcionales)	JWT
GET	/clientes/{id}	Consultar cliente por ID	JWT
PUT	/clientes/{id}	Actualizar datos del cliente	JWT + Activo
DELETE	/clientes/{id}	Baja lógica del cliente	JWT + Activo

Cuentas

Método	Endpoint	Descripción	Auth
GET	/cuentas/{numeroCuenta}	Consultar cuenta por número	JWT
GET	/cuentas/{numeroCuenta}/saldo	Consultar saldo de una cuenta	JWT

Usuarios

Método	Endpoint	Descripción	Auth
GET	/usuarios/{id}	Consultar información del usuario	JWT
PUT	/usuarios/{id}/password	Cambiar contraseña del usuario	JWT + Activo

Filtros Disponibles en GET /clientes

Query Param	Tipo	Ejemplo	Descripción
curp	String	?curp=PEGC950520HDFRRR01	Buscar por CURP exacto
rfc	String	?rfc=PEGC950520DFA	Buscar por RFC exacto
correo	String	?correo=cesar@mail.com	Buscar por correo electrónico
numeroCuenta	String	?numeroCuenta=1234567890	Buscar por número de cuenta

Query Param	Tipo	Ejemplo	Descripción
activos	Boolean	?activos=true	Solo clientes con usuario activo
fechaDesde / fechaHasta	ISO DateTime	?fechaDesde=2026-01-01T00:00:00&fechaHasta=2026-12-31T23:59:59	Rango de fechas de registro

Consultas Implementadas

Consulta Solicitada	Método Repository	Tipo de Query
Buscar cliente por CURP	findByCurp(String)	Spring Data (derivado)
Buscar cliente por RFC	findByRfc(String)	Spring Data (derivado)
Buscar cliente por correo	findByCorreo(String)	@Query JPQL personalizado
Consultar clientes activos	findClientesActivos()	@Query JPQL (usuario.activo=true)
Consultar saldo de una cuenta	findByNumeroCuenta(String)	Spring Data (derivado)
Clientes por rango de fechas	findByFechaRegistroBetween()	@Query JPQL con parámetros
Verificar CURP duplicada	existsByCurp(String)	Spring Data (derivado)
Verificar RFC duplicado	existsByRfc(String)	Spring Data (derivado)
Verificar correo duplicado	existsByCorreoElectronico(String)	Spring Data (derivado)

Reglas de Negocio Implementadas

#	Regla de Negocio	Implementación	Ubicación
1	Cliente mayor de 18 años	Period.between() calcula edad exacta	ClienteServiceImpl
2	CURP única	existsByCurp() + UNIQUE constraint en BD	Service + BD
3	RFC único	existsByRfc() + UNIQUE constraint en BD	Service + BD
4	Correo electrónico único	existsByCorreoElectronico() + UNIQUE	Service + BD
5	Teléfono de 10 dígitos	@Pattern(regexp="^\d{10}\$") + CHECK en BD	DTO + BD
6	Saldo inicial no negativo	CHECK (saldo >= 0) en BD	BD
7	Número de cuenta único	UNIQUE constraint + verificación en generación	Service + BD
8	Solo activos tienen cuentas activas	Baja lógica cambia cuentas a INACTIVA	ClienteServiceImpl
9	Contraseña segura	Regex: 8+ chars, mayus, minus, num, especial	DTO
10	Password cifrada con BCrypt	passwordEncoder.encode()	AuthServiceImpl
11	CURP, RFC y cuenta inmutables	Campos ausentes en ClienteActualizaDTO	DTO diseño
12	Baja lógica (no eliminar datos)	activo=false + cuentas=INACTIVA	ClienteServiceImpl
13	Congruencia CURP vs datos	Fecha y sexo cruzados con CURP	ClienteServiceImpl

Optimizaciones y Resiliencia

Se implementaron medidas adicionales para garantizar que el sistema sea robusto y resistente ante pruebas de estrés (JMeter) y ataques de denegación de servicio:

Mecanismo	Implementación	Propósito
Rate Limiting	RateLimitFilter.java con Bucket4j (20 req/seg por IP)	Protección contra DDoS y ataques de fuerza bruta
Connection Pool	HikariCP con maximum-pool-size=30	Evitar cuellos de botella en conexiones a BD
Paginación defensiva	PageRequest.of(0, 100) en findAll()	Evitar Out Of Memory al consultar miles de registros
Manejo de concurrencia	GlobalExceptionHandler + DataIntegrityViolation	Peticiones simultáneas devuelven 409 en lugar de 500
Transacciones atómicas	@Transactional en operaciones de escritura	Si algo falla, todo se revierte automáticamente

Tecnologías Utilizadas

Tecnología	Versión	Propósito
Java	17	Lenguaje de programación principal
Spring Boot	3.3.6	Framework de aplicación
Spring Data JPA	3.x	Acceso a datos con Hibernate ORM
Spring Security	6.x	Autenticación y autorización
PostgreSQL	16+	Base de datos relacional
Flyway	10.x	Migraciones versionadas de base de datos
JWT (jjwt)	0.11.5	Tokens de autenticación stateless
BCrypt	Spring Security	Cifrado de contraseñas
Lombok	1.18.34	Reducción de código boilerplate
MapStruct	1.6.3	Mapeo automático Entity ↔ DTO
Swagger/OpenAPI	2.2.0 (springdoc)	Documentación interactiva de la API
Bucket4j	8.7.0	Rate Limiting (Anti-DDoS)
HikariCP	Spring Boot	Pool de conexiones a base de datos
Gradle	9.6	Sistema de construcción y gestión de dependencias